Tabular Q-learning on Taxi

CPSC 4420 homework: OpenAI Gym / Gymnasium. Environment: Taxi. Code: [PASTE GITHUB REPO LINK HERE]

A. Environment description

Goal. The taxi starts somewhere on a 5x5 grid. It has to drive to the passenger, pick them up, drive to their destination, and drop them off. There are four special squares (R, G, Y, B) that can be a pickup or dropoff spot.

State space. One discrete number from 0 to 499: 25 taxi positions x 5 passenger locations (4 squares, or in the taxi) x 4 destinations. No discretization is needed, so the Q-table is 500 x 6.

Action space. 6 discrete actions: south, north, east, west, pickup, dropoff.

Transitions. Deterministic. Moving into a wall leaves the taxi where it is. Pickup only works if the taxi is on the passenger's square, and dropoff only works in the taxi at the destination.

Rewards. -1 every step, +20 for a successful dropoff, -10 for an illegal pickup or dropoff.

Termination. The episode terminates on a successful dropoff. It is truncated after 200 steps. In my update rule, only true termination zeroes the bootstrap term. Truncation does not, since it is just a time limit.

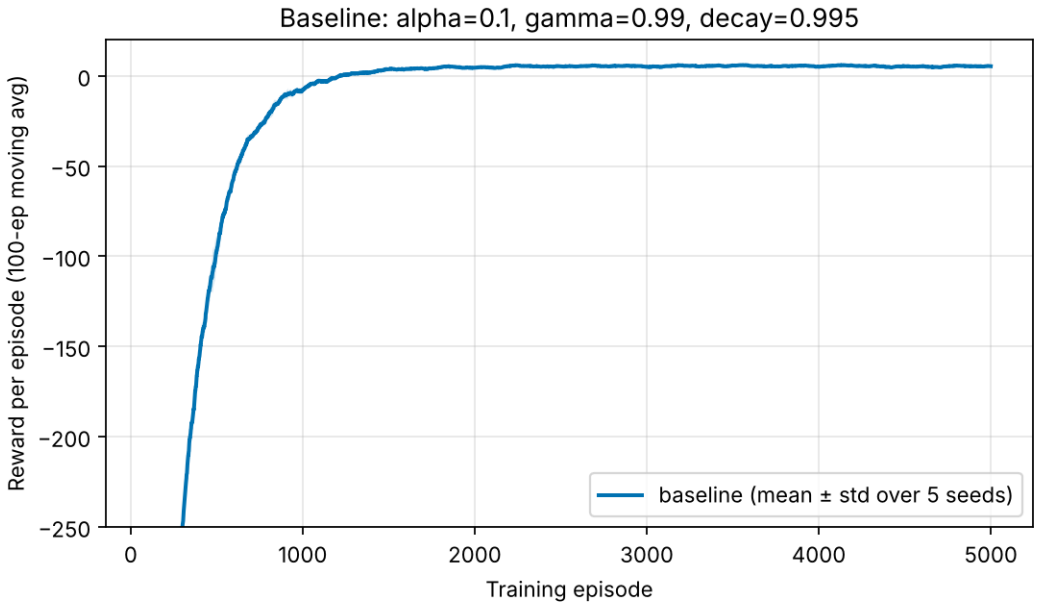
Settings. The installed Gymnasium (1.4.0) lists Taxi-v3 as deprecated, so I used Taxi-v4 (same task). Every curve is the mean of 5 training seeds (0-4), with a shaded band for one standard deviation across seeds. Curves are smoothed with a 100-episode moving average and the y-axis is clipped at -250, so the first few hundred episodes (which are around -800, close to the random-agent score) are cut off.

B. Q-learning implementation

The agent is in `qlearning.py`. The Q-table starts at all zeros. Actions are epsilon-greedy (ties in the greedy choice are broken randomly, so unvisited states do not always pick action 0). The update is $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max Q(s',a') - Q(s,a)]$, and the target is just r when the episode terminated. Epsilon is multiplied by a decay factor after every episode until it hits a floor. alpha, gamma, epsilon start, epsilon floor, decay, number of episodes, seed and noise level are all arguments of `train()`.

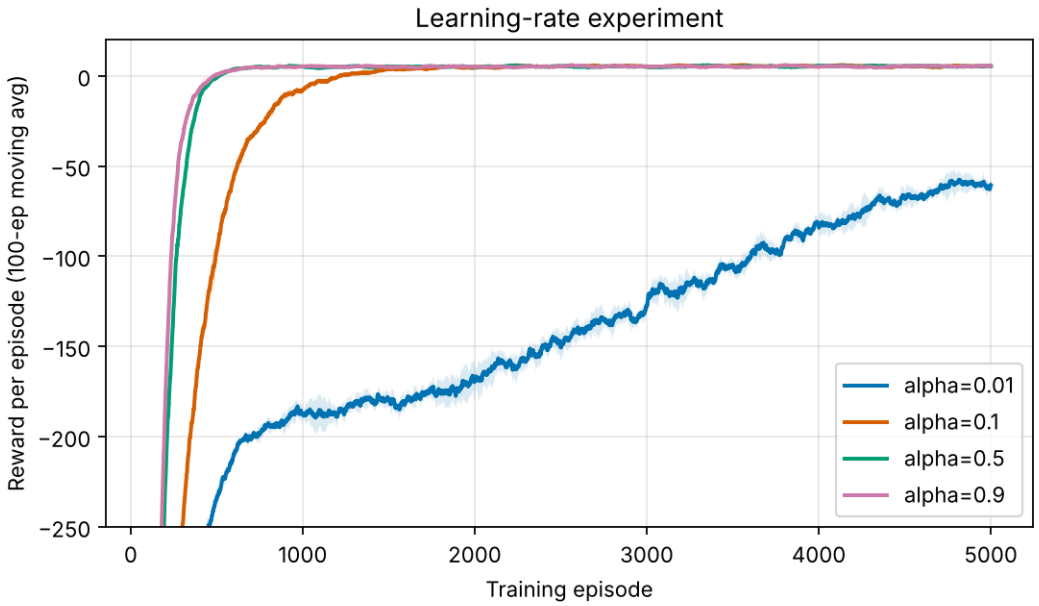
C. Baseline training

Baseline: alpha = 0.1, gamma = 0.99, epsilon starting at 1.0 and decaying by 0.995 per episode down to 0.05, 5000 episodes.



The agent starts out about as bad as a random policy (around -800 per episode, because it keeps hitting the 200-step limit and making illegal pickups). The moving average crosses zero after about 1211 episodes and flattens out near +5.2. That plateau is lower than the greedy score reported in G because training still takes a random action about 5% of the time (epsilon floor 0.05).

D. Learning-rate experiment

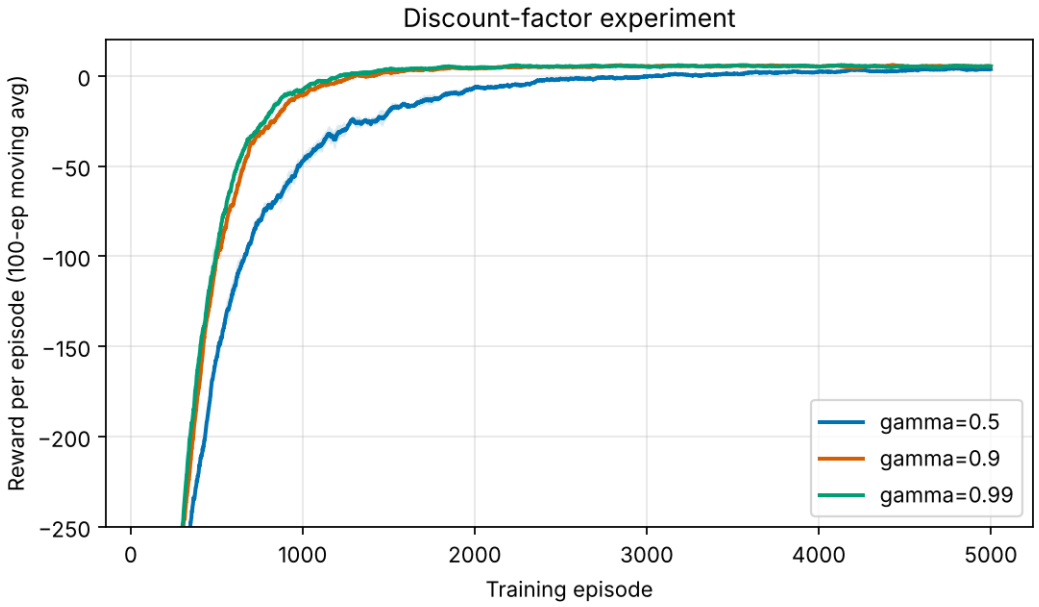


alpha	Episodes until avg > 0	Avg train reward (last 500)	Greedy eval reward	Greedy success rate
0.01	never	-61.78	-234.46	14%
0.1	1211	5.24	8.26	100%

0.5	510	5.31	8.20	100%
0.9	477	5.37	8.12	100%

(Greedy evaluation here is 100 episodes per seed on separate evaluation seeds, averaged over 5 training seeds.) Learning speed depends a lot on alpha. With 0.01 the agent is still far from solved after 5000 episodes (its greedy policy only completes 14% of episodes). 0.1 gets there in about 1200 episodes, and 0.5 and 0.9 get there in about 500. I did not see instability at the high learning rates, and the curves for 0.5 and 0.9 are smooth with a very thin band. That makes sense here because the environment is deterministic, so a big step toward the target does not chase noise. The final greedy rewards for 0.5 and 0.9 are slightly lower than for 0.1 (8.20 and 8.12 vs 8.26). The gap is small, and I did not dig into which states cause it. I would expect a high alpha to be a worse choice in a noisy environment, but I did not test that in the report.

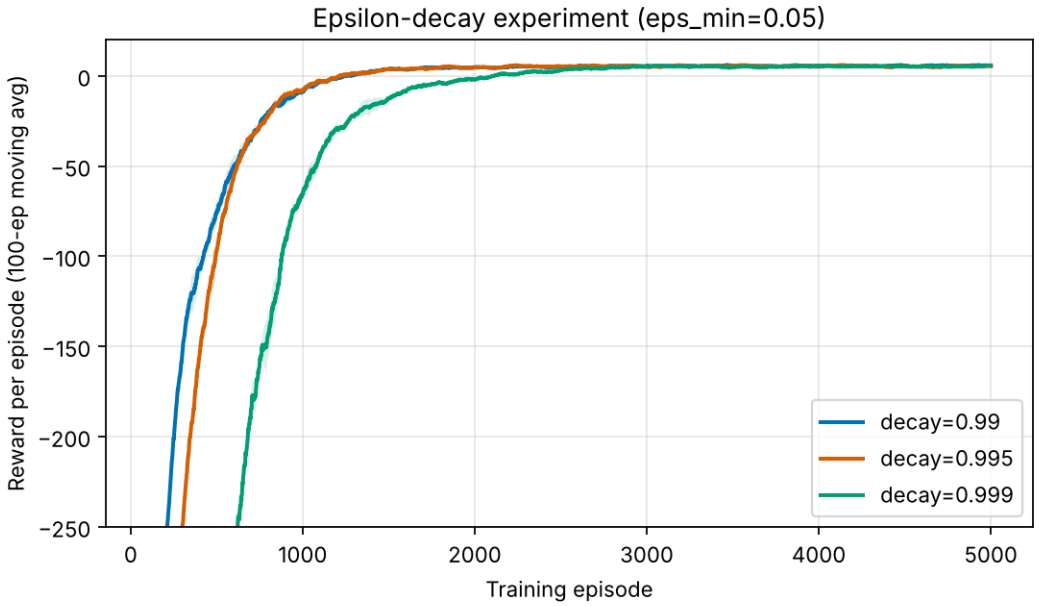
E. Discount-factor experiment



gamma	Episodes until avg > 0	Avg train reward (last 500)	Greedy eval reward	Greedy success rate
0.50	2639	3.61	-68.28	63%
0.90	1295	5.32	8.26	100%
0.99	1211	5.24	8.26	100%

gamma = 0.9 and 0.99 end up with the same greedy policy quality. A typical trip is only about 13 steps, and 0.9^13 is about 0.25, so the +20 at the end still shows up clearly in Q-values near the start. With gamma = 0.5 the +20 is discounted to almost nothing after 10 steps or so, so states far from the goal all look about the same to the agent. It learns more slowly and its greedy policy is bad: only 63% of greedy episodes finish and the mean reward is -68. Interestingly its training reward (3.6) looks fine by the end, much better than its greedy score. My guess is that the greedy policy gets stuck looping in places where Q-values are nearly flat, and the 5% random actions during training knock it out of those loops. I did not test that directly.

F. Additional hyperparameter: epsilon decay



Decay per episode	Episodes to reach eps=0.05	Episodes until avg > 0	Avg train reward (last 500)	Greedy eval reward
0.99	298	1261	5.64	8.26
0.995	598	1211	5.24	8.26
0.999	2994	2064	5.23	8.26

All three decays end with the same greedy policy (8.26, 8.26, 8.26). They differ in how long training takes. Decaying too slowly (0.999) keeps the agent acting randomly for about 3000 episodes, so it takes about 2060 episodes to get a positive average versus about 1210-1260 for the other two. The fastest decay (0.99) did not hurt in this environment, since Taxi is small and 300 episodes of mostly random play already visits most states. It also has the best training reward at the end, since it spends the most episodes at the epsilon floor. In a bigger environment I would expect the fastest decay to risk locking in a bad policy.

G. Evaluation of the best agent

How I picked the best configuration. For every configuration I trained 5 seeds and ran the final Q-table greedily (epsilon = 0) for 100 episodes each on a set of evaluation seeds (start states) used only for selection. I picked the configuration with the highest mean greedy reward. Several configurations tie almost exactly (baseline, gamma = 0.9, decay = 0.99, and decay = 0.999 are all at 8.26 or within 0.01), so honestly the choice among them does not matter much. I kept the baseline, which is also the first in the tie. If speed of learning mattered more, alpha = 0.5 would be a fair choice (about 2.4x faster to reach a positive average at a cost of 0.06 reward).

Configuration	Greedy reward	Success	Steps	Seed std of mean reward
baseline	8.26	100%	12.74	0.00
alpha=0.01	-234.46	14%	172.93	40.33

alpha=0.5	8.20	100%	12.80	0.03
alpha=0.9	8.12	100%	12.88	0.04
gamma=0.5	-68.28	63%	81.51	4.90
gamma=0.9	8.26	100%	12.74	0.00
decay=0.99	8.26	100%	12.74	0.00
decay=0.999	8.26	100%	12.74	0.01

Final evaluation. I then evaluated the chosen agent on a fresh set of start states (different seeds from selection): 5 trained agents x 100 episodes = 500 episodes with exploration off. Mean reward **7.97** (std 2.26 across episodes), success rate **100%**, mean steps to finish **13.03**. The score is a bit lower than the 8.26 from the selection seeds because the start states are different (the agent's policy is the same). All 5 seeds scored the same here (7.97), so they learned equivalent policies.

Comparison. A random policy scores about -766 with a 5% success rate (196 steps on average), and an untrained Q-table (always action 0) scores -200 and never finishes. The trained agent solves every episode in about 13 steps. Since the best configuration is the baseline, the comparison to the baseline is the comparison above: the other settings either tie it, learn more slowly, or are clearly worse (alpha = 0.01, gamma = 0.5). The video `results/trained_agent.mp4` shows this agent on 3 episodes.

Conclusions

Tabular Q-learning solves Taxi reliably (100% success, about 13 steps). Learning rate mattered most for speed (0.01 is too slow, 0.5 to 0.9 is fastest in this deterministic task). A low discount factor (0.5) breaks learning because the final reward gets discounted away, while 0.9 and 0.99 both work. Epsilon decay changed only training time.